

oops

Linux Error Explainer — TUI Tool

Product Requirements Document

Version 1.0 • Open Source • Pure Python • MIT License

1. Product Overview

Product Name: oops

Type: Open Source TUI (Terminal User Interface) Tool

Language: Python 3.8+

License: MIT

Target Platform: Linux (Ubuntu, Fedora, Arch, Debian and derivatives)

Status: Pre-development — v1.0 Scope Definition

1.1 Problem Statement

Linux beginners regularly encounter cryptic error messages in the terminal. Their typical response is to copy the error into a browser search, sift through outdated Stack Overflow answers, and try fixes blindly — often making things worse. There is no tool that reads the error in context and explains it immediately, in plain English, inside the terminal itself.

1.2 Solution

oops is a command you run immediately after a failing command. It reads your last error from shell history and stderr, matches it against a curated pattern library, and opens a clean TUI screen showing:

- What went wrong — in plain English
- Why it happened — root cause, not symptoms
- Exactly what command to run to fix it — copyable with one keypress
- A short educational note explaining the concept behind the error

1.3 Elevator Pitch

"You ran a command. It broke. You typed oops. Now you know what happened and how to fix it — without leaving the terminal."

2. Target Users

2.1 Primary User

Linux newcomers — people who have recently switched from Windows or macOS, CS students in their first year, or anyone self-teaching Linux through tutorials. These users understand that something went wrong, but lack the vocabulary and mental model to interpret error output.

2.2 Secondary Users

- Intermediate users who occasionally encounter unfamiliar errors (e.g., obscure systemd or network errors)
- Educators and bootcamp instructors who want to reduce friction for students
- Contributors who want to expand the pattern library for their distro or use case

2.3 Anti-Users (Not Building For)

- Senior sysadmins and DevOps engineers — they don't need error explanations
- Users on macOS or Windows WSL (not the initial target, though contributions welcome)

3. Goals & Success Metrics

3.1 Goals

- Reduce the time a Linux beginner spends understanding a common error from 5-10 minutes to under 30 seconds
- Cover the top 50 most common Linux errors in v1.0
- Build a community-extensible pattern library where contributors can add errors via simple JSON PRs
- Achieve a memorable, minimal UX — one command, one screen, one fix

3.2 Success Metrics

- GitHub: 200+ stars within 3 months of launch
- Pattern library: 20+ community-contributed patterns within 60 days
- PyPI: 500+ installs in the first month
- Reddit r/linuxquestions or r/linux4noobs feature or mention

4. Feature Specification

4.1 Core Features — v1.0

Feature	Priority	Notes
Auto error capture	P0	Reads last command + stderr from shell history automatically
Pattern matching engine	P0	Regex-based matching against patterns.json library
TUI result screen	P0	Built with Textual — shows error, cause, fix, learn more
One-key copy of fix	P0	Press 'c' to copy the fix command to clipboard
20 core error patterns	P0	apt, pip, chmod, sudo, port-in-use, command-not-found, etc.
Distro detection	P1	Detect Ubuntu vs Arch vs Fedora, show distro-appropriate fix
patterns.json contributor format	P1	Clean JSON schema for community to add new patterns easily
pip install support	P1	Installable via pip install oops-cli
Shell integration (bash/zsh)	P1	Optional alias so oops auto-captures stderr without extra setup
Unknown error fallback	P1	Graceful message + Google search link when pattern not found
Color themes	P2	Light / dark / high contrast TUI themes
Error history log	P2	Local ~/.oops/history.json of past errors and fixes applied

4.2 TUI Screen Layout

The TUI result screen is organized into four panels stacked vertically:

	WHAT WENT WRONG
Plain English description of the error	
	WHY IT HAPPENED
Root cause explanation (1-2 sentences)	
	HOW TO FIX IT
\$ the-exact-command --to-run-it [press C to copy]	
	LEARN MORE
Short concept note so the user understands it next time	

5. Technical Architecture

5.1 Stack

Python 3.8+	Textual	psutil	pyperclip	re	subprocess
-------------	---------	--------	-----------	----	------------

- Textual — TUI rendering framework (modern, beautiful, well-maintained)
- psutil — process and system info where needed
- pyperclip — clipboard access for one-key copy
- re — regex engine for pattern matching
- subprocess — shell history and journalctl access
- No internet required — fully offline, no API calls, no telemetry

5.2 Project Structure

```
oops/  oops/  __init__.py  cli.py  # Entry point - `oops` command
capture.py  # Reads shell history + stderr  matcher.py  # Matches error
against patterns  tui.py  # Textual TUI screen  distro.py  #
Detects distro for fix variants  patterns/  core.json  # 20 core error
patterns  community.json  # Community contributed patterns  tests/  README.md
CONTRIBUTING.md  pyproject.toml
```

5.3 Pattern Schema (patterns.json)

Each error pattern follows this JSON structure, making community contributions simple and reviewable:

```
{  "id": "apt-unable-to-fetch",  "match": ["Unable to fetch some archives", "apt-get
update"],  "what": "apt cannot download packages right now.",  "why": "Your local
package list is outdated or offline.",  "fix": {"default": "sudo apt update && sudo
apt install {pkg}"},  "learn": "apt caches a list of available packages locally...",
"tags": ["apt", "package-manager", "ubuntu", "debian"] }
```

6. v1.0 Error Pattern Coverage

The following 20 error patterns form the core library for launch:

Error Pattern	Category
E: Unable to fetch some archives	Package Manager (apt)
E: Could not get lock /var/lib/dpkg/lock	Package Manager (apt)
ModuleNotFoundError: No module named 'x'	Python / pip
pip: command not found	Python / pip
Permission denied (chmod)	File Permissions

sudo: command not found	sudo / privileges
command not found: xyz	PATH / Installation
Address already in use (port)	Networking / Ports
Connection refused	Networking / Services
No space left on device	Disk Space
bash: ./script.sh: Permission denied	File Permissions
git: 'push' failed: repository not found	Git
CONFLICT (content): Merge conflict	Git
systemctl: Unit xyz.service not found	systemd / Services
Failed to connect to bus: No such file	systemd / dbus
WARNING: Running pip as the root user	pip / sudo
virtualenv: command not found	Python environments
zsh: command not found (after install)	Shell / PATH refresh
tar: Error opening archive	File Archiving
SSH: Host key verification failed	SSH / Security

7. Development Roadmap

Phase	Timeline	Deliverables
v0.1	Weekend 1	Project scaffolding, capture.py, 5 patterns, basic TUI renders
v0.2	Weekend 2	All 20 core patterns, distro detection, pyperclip copy feature
v0.3	Week 3	Shell integration (bash/zsh alias), pip packaging, README
v1.0	Week 4	CONTRIBUTING.md, community.json schema, PyPI publish, GitHub launch
v1.1	Month 2	Community patterns, error history log, unknown error fallback
v2.0	Month 3+	Optional AI mode (local LLM or API), extended distro support

8. Open Source Strategy

8.1 Why Open Source Works Here

- Error patterns are crowdsourced knowledge — the community knows their distro's errors better than any single author
- JSON pattern contributions are beginner-friendly first PRs, making oops a great project for new contributors
- The Linux community has strong values around open tooling — a closed version would get no traction

8.2 Community Contribution Model

- CONTRIBUTING.md will clearly document the pattern JSON schema
- A pattern submission template will be provided for GitHub Issues
- All patterns reviewed for accuracy before merging to core.json
- Contributors credited in README and releases

8.3 Distribution

- Primary: pip install oops-cli (PyPI)
- Secondary: AUR package (Arch Linux community)
- Future: apt/dnf package if adoption justifies it

9. Out of Scope (v1.0)

- AI/LLM integration — pure Python pattern matching only in v1
- macOS or Windows support — Linux-first
- GUI or web interface — TUI only
- Automatic command execution — oops only suggests, never auto-runs fixes
- Telemetry or data collection — fully offline and private

10. Risks & Mitigations

Risk	Likelihood	Mitigation
Error capture varies across shells	High	Support bash + zsh first, document zsh wrapper clearly
Pattern library grows stale	Medium	Community PR model, auto-link GitHub Issues for missing errors
Similar tool already exists	Low	Research confirms no focused beginner TUI error explainer exists

Textual introduces breaking changes	Low	Pin Textual version, monitor releases
-------------------------------------	-----	---------------------------------------

Built with ❤️ for the Linux community
oops — because everyone deserves to understand their errors